

30. Vyhledávání a řazení

30. Vyhledávání a řazení

SZZ okruh 30 (FIT BUT). Klasické řadící a vyhledávací algoritmy a jejich vlastnosti.

Shrnutí

Řazení

- **Vlastnosti:** **stabilita** (zachová pořadí stejných klíčů), **přirozenost** (rychlejší na téměř seřazených datech), **in situ** (bez pomocné paměti), časová/prostorová složitost.
- Principy: výběr (Selection, Bubble), vkládání (Insertion), rozdělování (Quick, Shell), slučování (Merge), třídění (Radix).
- Složitosti: Bubble/Selection/Insertion $O(n^2)$; Quick $O(n \log n)$ průměrně (nejhorší $O(n^2)$); Merge/Heap $O(n \log n)$; Radix $O(n \cdot d)$.
- Stabilní: Bubble, Insertion, Merge, Radix. Nestabilní: Selection, Quick, Heap, Shell.
- Porovnávací řazení nelze rychleji než $O(n \log n)$; Radix/counting obchází (nesrovnává).

[!note] Ke kontrole Zdroj označuje **Merge sort** jako „in situ” s prostorovou složitostí $O(\log n)$. Standardní Merge sort **není in-place** — vyžaduje $O(n)$ pomocné paměti na slučování ($O(\log n)$ je jen rekurzní zásobník). Ověř formulaci.

Vyhledávání

- **Sekvenční** (neseřazené $O(n)$; se zarážkou; seřazené — rychlejší jen neúspěšné; adaptivní dle četnosti).
- **Binární** v seřazeném poli $O(\log n)$ (problém s vkládáním/mazáním); **BVS** (snazší vkládání/mazání); **stromy s více klíči** (B/B+ stromy — DB, FS).
- **Hash tabulka** — ideálně $O(1)$, v nejhorším $O(n)$; explicitní/implicitní řetězení synonym.

Vyhledávání v textu

- **Naivní** $O(m \cdot n)$; **Knuth-Morris-Pratt** (konečný automat, $O(m+n)$); **Boyer-Moore** (porovnává zprava, velké skoky); **Rabin-Karp** (hashování okénka); **Aho-Corasick** (více vzorků, písmennový strom).

Datové struktury viz [[okruhy/29-datove-ridici-struktury]]; asymptotická složitost viz [[okruhy/32-slozitost-algoritmu]]; KMP a automaty viz [[okruhy/21-regularni-jazyky]].

Související syntéza

- [[synthesis/slozitost-napric-algoritmy|Asymptotická složitost napříč algoritmy]] — syntéza
- [[synthesis/vyhledavaci-b-stromy|Vyhledávací stromy × paměťová hierarchie]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Vyhledávání □ [[#Vyhledávání]] - Typy a složitosti? □ Sekvenční $O(n)$, binární (seřazené pole) $O(\log n)$, BVS $O(\log n)$ vyvážený, hash $O(1)$ ideálně. - Podmínka binárního vyhledávání? □ Seřazené pole (relace uspořádání). - Kolize v hash tabulce? □ Implicitní (otevřená adresace) × explicitní (zřetězení seznamem/stromem).

Řazení □ [[#Řazení]] - QuickSort vs. MergeSort vs. HeapSort? □ Quick $O(n \log n)$ prům. (nejhorší $O(n^2)$, in situ + $O(\log n)$ zásobník); Merge $O(n \log n)$ stabilní; Heap $O(n \log n)$ in situ nestabilní. - Stabilita / přirozenost / in situ? □ Stabilní zachová pořadí stejných klíčů; přirozený rychlejší na seřazených; in situ bez pomocné paměti. - Rychleji než $O(n \log n)$? □ Porovnávací ne; Radix/counting ano (nesrovnávají, využívají strukturu klíče).

Vyhledávání v textu □ [[#Vyhledávání v textu]] - KMP / Boyer-Moore? □ KMP konečný automat, nevrací se v textu ($O(m+n)$); Boyer-Moore porovnává zprava □ velké skoky.

Plné znění (ke studiu)

Řazení

- **Třídění (sorting)** položek neuspořádané množiny je **uspořádání do tříd podle** hodnoty daného **atributu** – klíče položky. **Mezi třídami nemusí být definovaná relace uspořádání** (jablka, hrušky, švestky lze třídit). Často se používá **řazení (ordering) pro třídění**.
- **Řazení (ordering, sequencing)** je **uspořádání** položek podle **relace lineárního uspořádání** nad klíči. Termín se nepoužívá často.
- **Setřídění (merging)** je vytváření souboru seřazených položek **sjednocením** několika souborů položek téhož typu, které **jsou již seřazené**. Příkladem je algoritmus Merge sort.

Vlastnosti řadících algoritmů

Slouží pro výběr vhodného algoritmu pro konkrétní implementaci.

Přirozenost

Algoritmus se chová přirozeně pokud:

- doba potřebná k seřazení **náhodně uspořádaného** pole **je větší než** k seřazení již **uspořádaného pole**,
- doba potřebná k seřazení **opačně seřazeného** pole **je větší než** doba k seřazení **náhodně uspořádaného** pole.

Jinak říkáme, že se algoritmus nechová přirozeně.

Stabilita

Stabilita vyjadřuje, zda mechanismus algoritmu zachovává **relativní pořadí klíčů se stejnou hodnotou**. Sekvence 7,5',3,1,5'',9,2,5''',8,4,6 pro následující příklad.

- **stabilní** algoritmus: 1,2,3,4,5',5'',5''',6,7,8,9.
- **nestabilní** algoritmus: 1,2,3,4,5'',5',5''',6,7,8,9 (nebo jinak).

Algoritmy řazení

Podle principu řazení je dělíme na:

- Princip **výběru (selection)** – **přesouvají maximum/minimum** do výstupní posloupnosti. Např.: **Selection sort**, **Bubble sort** a jeho modifikace (**Ripple sort**, **Shaker sort**, **Shuttle sort**)
- Princip **vkládání (insertion)** – **vkládají** postupně prvky **do seřazené** výstupní **posloupnosti**. Např.: **Insertion sort**, ...
- Princip **rozdělování (partition)** – **rozdělují** postupně množinu prvků na **dvě podmnožiny** tak, že prvky **jedné jsou menší než prvky druhé**. Např.: **Quick sort**, **Shell sort**, ...
- Princip **slučování (merging)** – **setřídí** se postupně **dvě seřazené posloupnosti** do jedné. Např.: **Merge sort**, ...
- **Jiné principy**. Např.: **řazení tříděním**, ...

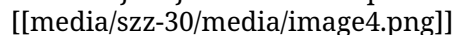
Podle typu procesoru:

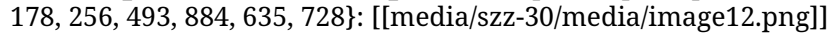
- **sériové** (jeden procesor) – jedna operace v daném okamžiku,
- **paralelní** (více procesorů) – více souběžných operací

Podle přístupu k paměti:

- **přímý (náhodný)** přístup – metody vnitřního řazení (řazení polí)
- **sekvenční** přístup – metody vnějšího řazení (řazení souborů a seznamů)

Řazení tříděním - Radix sort

Řazení tříděním musí probíhat od **nejnižší po nejvyšší** prioritu (**od LSB po MSB**, od jednotek přes desítky, stovky, tisíce, ... tj. podle základu). Jedná se o **třídění pomocí přihrádek** (u desítkového základu jich je 10 a třídění provádíme pro každou číslici klíče). Využití pro řazení děrných štítků. 

Příklad **správného** (vlevo) a **špatného** (vpravo) postupu řazení tříděním na množině {342, 835, 942, 178, 256, 493, 884, 635, 728}: 

Vlastnosti:

- **stabilní**,
- **nechová se přirozeně**,
- **nepracuje in situ**,
- **$O(n \cdot d)$** časová složitost, kde n je počet prvků v poli a d je počet číslic v největším čísle.

Selection sort

Princip metody je postaven na **nalezení extrémního prvku** v zadaném segmentu pole a jeho **výměna na konec (začátek) seřazené části pole**. Příklad řazení od největšího k nejmenšímu, viz <https://www.algoritmy.net/article/4/Selection-sort>:

1. (3 2 8 7 6) - zadané pole,
2. (3 2 8 7 6) - nejvyšší číslo je 8, prohodíme ho tedy s číslem 3 na indexu **0**,
3. (8 2 3 7 6) - nejvyšší číslo je 7, prohodíme ho tedy s číslem 2 na indexu **1**,
4. (8 7 3 2 6) - nejvyšší číslo je 6, prohodíme ho tedy s číslem 3 na indexu **2**,

5. (8 7 6 2 3) - nejvyšší číslo je 3, prohodíme ho tedy s číslem 2 na indexu **3**,

6. (8 7 6 3 2) - seřazeno

Vlastnosti:

- **nestabilní** - z důvodu výměny prvků,
- měla by být **přirozená**,
- složitost: **kvadratická $O(n^2)$** ,
- pracuje **in situ**.

Bubble sort

Stejně jako selection sort pracuje metoda na principu **nalezení extrémního prvku** a jeho **umístění na konec (začátek) již seřazené části**. Liší se ale v principu **nalezení** extrému **výměny** prvků, viz <https://www.algoritmy.net/article/3/Bubble-sort>.

Postup:

- Porovnává se každá dvojice a v případě **obráceného uspořádání** (záleží, jestli řadíme od největšího nebo od nejmenšího) **se přehodí**.
- Při pohybu **zleva doprava a řazení od nejmenšího k největšímu** se tak **maximum** dostane **na poslední pozici**. **Minimum** se posune o jedno místo **směrem** ke své **konečné pozici**.

Vlastnosti:

- **stabilní** - na rozdíl od selection sort,
- **přirozená** - nejrychlejší metoda pro již seřazené pole,
- složitost: **kvadratická $O(n^2)$** ,
- pracuje **in situ**.

Další **odvozené metody** od Bubble sort (Ripple sort, Shaker sort, Shuttle sort) jsou obecně rychlejší, ale ne z pohledu **časové složitosti** (zůstává **$O(n^2)$**).

Heap sort

Princip algoritmu je založený na **hromadě**. Hromada je struktura **stromového typu**, pro niž platí, že mezi **otcovským** uzlem a **všemi jeho synovskými** (to platí i v **libovolných podstromech**) uzly platí **stejná relace uspořádání** (buď je menší, nebo větší). Nejčastěji se používá **binární hromada**, která je založena na **binárním stromu**, který je konstruován tak, aby:

- všechny hladiny kromě poslední **byly plně obsazené**,
- **poslední** hladina je zaplněna **zleva**.

Binární hromadu lze také použít např. k implementaci prioritní fronty (pokud neexistují prvky se stejnou prioritou a stačí nám přístup k nejprioritnějšímu prvku).

Vytvoření hromady [[media/szz-30/media/image11.png]]

Rekonstrukce hromady Rekonstrukce **hromady** se realizuje pomocí **prosetí/zatřesení** (sift), která **znovuustanoví** hromadu **porušenou pouze v kořeni** Heap sort in 4 minutes:

- na místo kořenového uzlu přesuneme **nejnižší a nejpravější uzel**,

- zatřese se s hromadou, což způsobí **propadnutí se prvku v kořeni** na místo, kam patří a **vypropagování největšího prvku do kořene**. Děje se tak **porovnáním s levým a pravým synem** a **výměnou** za toho **většího/menšího** (podle toho, jak řadíme).

Princip algoritmu

1. **Vytvoření hromady** (lze uchovávat v poli),
2. **Odebrání kořene** a umístění jej na konec pole (a zmenšení hromady, uspořádanou část pole již ignorujeme).
3. Pokud není hromada prázdná, **prosetí/zatřesení** s hromadou a pokračování s bodem 2, jinak máme seřazeno.

Vlastnosti:

- **nestabilní**,
- **nechová se přirozeně**,
- **in situ**,
- **linearitnická složitost $O(n \cdot \log n)$** .

Insertion sort

Pracuje na **principu řazení karet v ruce**: vyberu neseřazenou a vložím ji na místo, kam patří (zbytek posunu doprava, abych pro ni udělal místo). Princip algoritmu:

1. pole dělíme na **seřazenou část** (levou) a **neseřazenou část** (pravou), na začátku tvoří seřazenou část jeden prvek.
2. Z neseřazené části **vyber první prvek**.
3. V seřazené části **najdi jeho umístění** (místo, kde jeho pravý prvek je větší/menší a jeho levý prvek je menší/větší, případně jsou stejné)
4. **posuň prvky** vpravo od nalezené pozice k pozici umísťovaného prvku **o 1**.
5. Umísti tento prvek.
6. Pokračuj, dokud není neseřazená posloupnost prázdná.

Vlastnosti:

- **stabilní**,
- **přirozený**,
- **in situ**,
- **kvadratická složitost $O(n^2)$** .

Bubble-insert sort Modifikace, u které je v průběhu porovnávání prováděn přesun položek doprava výměnou za porovnávaný prvek.

Binary-insert sort Modifikace, u které je vyhledávání prováděno binárním způsobem (půlení intervalů). Pro zajištění stability metody musí binární vyhledávání při více stejných hodnotách vybrat místo za nejpravějším výskytem (Dijkstrova metoda).

<https://www.algoritmy.net/article/8/Insertion-sort>

Quick sort

Algoritmus fungující na principu **rozděl a panuj**. Jedná se o jeden z **nejrychlejších** algoritmů pro řazení (jeho rychlost je ale nedeterministická). Princip algoritmu:

1. rozděl množinu prvků na dvě podmnožiny tak, aby:
 1. první obsahovala prvky **menší/větší** nebo rovny **mediánu**,
 2. druhá obsahovala prvky **větší/menší** nebo rovny **mediánu**.
2. Takto získané podmnožiny opět rozděluj na další podmnožiny a obdobně uspořádávej prvky.
3. Skonči, pokud množina obsahuje pouze 1 prvek.

Přesun prvků podle mediánu je realizován následovně:

- Procházíme pole současně zleva a zprava.
- Zleva hledáme prvek větší nebo roven mediánu, zprava prvek menší nebo roven mediánu.
- Nalezené prvky vyměníme a hledáme další prvky pro výměnu.
- Proces ukončíme až se dvojice indexů překříží.

Medián je prvek, který dělí množinu na dvě části tak, že platí:

- nejméně 50 % hodnot je menších nebo rovných mediánu,
- nejméně 50 % hodnot je větších nebo rovných mediánu.

Samotné vyhledání mediánu je časově náročné, takže se používá **pseudomedián**, což může být **náhodná hodnota** z množiny nebo hodnota, která je ve **středu** množiny na **indexu $(\text{left} + \text{right})/2$** , případně lze vybrat medián ze tří prvků (na začátku, konci a uprostřed, či jinak).

Vlastnosti:

- **nestabilní**,
- **nepracuje přirozeně**,
- **linearity** časová složitost - v **nejhorším** případě je složitost **kvadratická**,
- pracuje **in situ**, ALE **potřebuje zásobník** pro ukládání hranice ještě nezpracovaných částí. Důležitý trik pro zmenšení zásobníku je **na zásobník uložit (pouze) tu větší část** a dále, tedy dříve, **zpracovávat vždy menší** z rozdělených částí. To zaručí, že stačí zásobník velikosti **$\log_2(n)$** , tj. například pro miliardu prvků stačí hloubka 30 položek. Bez této optimalizace by byla paměť potřebná pro zásobník **až lineární**, takhle je logaritmická **$O(\log(n))$** .

<https://www.algoritmy.net/article/10/Quicksort>

Shell sort

Algoritmus fungující na principu „**rozděl a panuj**“. Vytváří sekvence prvků, které seřazuje pomocí insertion sortu. Princip:

1. Rozděl vstupní pole na posloupnosti o **délce 2** (prvky posloupnosti musí být co nejdále od sebe),
2. Každou posloupnost seřaď pomocí insertion sort,
3. Rozděl vstupní pole na posloupnosti o **délce 4** (prvky posloupnosti musí být co nejdále od sebe),
4. ...
5. Rozděl vstupní pole na posloupnost o **délce pole** a seřaď je pomocí insertion sortu.

nejlepší délky posloupností - 1, 4, 10, 23, 57, 132, 301, 701, 1750.

Vlastnosti:

- **nestabilní**,
- měla by být **přirozená**,
- pracuje **in situ**,
- časová složitost je v nejhorším případě **kvadratická**, ale vybráním vhodné řady lze snížit na **$O(n^{3/2})$** nebo **$O(n \cdot \log n)^2$** .

<https://www.algoritmy.net/article/154/Shell-sort> 

Merge sort

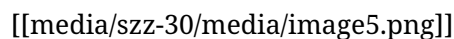
Další z algoritmů typu **rozděl a panuj**. Princip:

1. Postupně **půlíme** pole (v případě lichého počtu má jedna část o prvek navíc), dokud není o velikosti 1.
2. Poté **spojujeme** vždy dvě sousední pole tak, aby **vzniklo jedno seřazené** pole. Toto seřazování je jednoduché, protože již **spojujeme seřazená pole**, tudíž máme dva ukazatele do těchto polí a vždy vybíráme prvek, který je mezi poli **nejmenší/největší**.

Vlastnosti:

- **stabilní**,
- spíš není přirozená, vždy se musí dělit a poté setřídovat,
- **in situ**, ale prostorová složitost je **logaritmická $O(\log(n))$** pro uchovávání hranic polí po dělení - lze použít zásobník, ale jednodušší je použití **rekurze**,
- **lineární $O(n \cdot \log n)$** časová složitost.

<https://www.algoritmy.net/article/13/Merge-sort>



Shrnutí



Řazení dle více klíčů

Lze řešit opakovaným řazením, které se podobá se radixovému (přihrádkovému) řazení např. pro den, měsíc a rok. Musí platit:

- řadíme **od klíče s nejmenší** prioritou **po klíč s největší** prioritou,
- nutné použít **stabilní řadící metodu**.

Druhou možností je provádět řazení **pouze jednou**, ale současně **porovnávat všechny klíče** počínaje od toho s největší prioritou. Z čehož plyne třetí možnost, a to sloučit klíče do jednoho (aglomerovaný klíč). U příkladu se dny, měsíce a roky se jedná o rodné číslo bez posledních 4 znaků - RRMDD (**pozor** ženy mají **MM zvýšené o 50**).

Řazení bez přesunu položek

Vhodné v případech, kdy řadíme **velké objekty** a jejich **přesuny v paměti** jsou **drahé**. K tomu, abychom položky museli přesunovat, používáme **pomocné pole - pořadník**. Po dokončení řazení **pořadník udává**, v jakém **pořadí** by měly být seřazeny **položky původního pole**. Tj. na první pozici pořadníku je index prvního prvku seřazeného pole atd. [[media/szz-30/media/image6.png]]

Pole seřazené pomocí **pořadníku** lze také **zřetězit**, v jednotlivých prvcích musí být **vyhrazená paměť** pro **index následujícího prvku**. Zřetěžené prvky pak lze **procházet sekvenčně principem spojových seznamů**, nebo je lze **převést do seřazeného pole** (to je možné i bez zřetěžení podle pořadníku - MacLarenův algoritmus).

Vyhledávání

Vyhledávání lze realizovat jako:

- sekvenční vyhledávání v **neseřazeném poli**,
- sekvenční vyhledávání v **neseřazeném poli se zarážkou**,
- sekvenční vyhledávání v **seřazeném poli**,
- sekvenční vyhledávání v poli **seřazeném podle pravděpodobnosti vyhledání klíče** (nejpravděpodobnější klíče jsou na začátku pole),
- sekvenční vyhledávání v poli s **adaptivním uspořádáním podle četnosti vyhledání** (prvky se s četností vyhledávání přesouvají k začátku pole),
- **binární vyhledávání v seřazeném poli**,
- **binární vyhledávání pomocí stromů**,
- vyhledávání pomocí **stromů s více klíči ve vrcholech**,
- vyhledávání pomocí tabulky s rozptýlenými položkami (hash table).

Při vyhledávání sledujeme doby vyhledání (**kritéria**) při **úspěšném/neúspěšném** vyhledávání.

- **minimální**,
- **maximální**,
- **průměrná/střední**.

Sekvenční vyhledávání

Prvky pole se prochází jeden po druhém.

V neseřazeném poli

Nejrychleji jsou nalezeny položky na počátku vyhledávání. Jednoduchá implementace (**procházení polem a test na hodnotu**).

- **minimální** čas úspěšného vyhledání = **1**
- **maximální** čas úspěšného vyhledání = **N**
- **průměrný** čas úspěšného vyhledání = **N/2**
- Čas **neúspěšného** vyhledání = **N**

V neseřazeném poli se zarážkou

Na konci struktury je “**zarážka**”, což je **hledaný klíč**, tedy hodnota je vždy nalezena. Toto přidává **urychlení** v tom, že není potřeba po každém porovnání klíčů **testovat konec struktury**, pouze se otestuje na konci vyhledávání jestli se **nalezla zarážka nebo hledaná položka**.

V seřazeném poli

Jakmile se narazí na hodnotu klíče, která je **větší než hledaný klíč**, vyhledávání se ukončuje jako **neúspěšné**. Tato metoda se **urychlí pouze pro neúspěšné hledání**. Vyhledávání v seřazeném poli lze ale urychlit např. **půlením intervalů**, viz dále. Funguje pouze pro klíče, u kterých **existuje relace uspořádání**. Problematické jsou operace vkládání a odebírání prvků, je nutné pole **znovu seřadit**, respektive **posunout segment pole v paměti**. Odebírání lze řešit **zaslepením** (označením indexu za nevyužitý, respektive nastavení jeho hodnoty na hodnotu, která nebude hledána).

Podle četnosti přístupu (adaptivní vyhledávání)

Nejčastěji hledané položky **jsou na začátku struktury - pole/seznamu** (používá se k tomu **počítadlo vyhledávání** a jednou za čas je pole podle počítadla seřazeno). Druhou variantou je, že po nalezení položky se **nalezená položka prohodí se svým levým** sousedem (není potřeba počítadlo, nejčastěji hledané položky se tak přesouvají k začátku struktury) - **adaptivní rekonfigurace podle četnosti vyhledávání**. Postupy lze využít i u prvků, které mezi sebou nemají definovanou relaci uspořádání.

Podle pravděpodobnosti

Jedná se o podobný případ viz sekce výše, kde pravděpodobnosti vyhledávání jsou známe předem a podle toho je pole seřazené (poté se již uspořádání nemění).

Nesekvenční vyhledávání

Přístup k hledaným položkám může být náhodný.

Binární vyhledávání v poli (seřazeném)

Provádí se nad **seřazenou množinou klíčů** s náhodným přístupem (pole). Metoda připomíná metodu **půlení intervalů**. **Složitost** je při nejhorším **logaritmická $O(\log n)$** . Pole **půlíme** a podle **prostřední hodnoty** prohledáváme **levou nebo pravou část**. Opět je zde stejný **problém s vkládáním a odebíráním prvků** z pole.

Dijkstrova varianta binárního vyhledávání

Vychází z předpokladu, že v poli může být **více položek se shodným klíčem**. Hledá se tedy **nejpravější nebo nejlevější klíč** (ostatní pak lze získat jednoduše sekvenčním průchodem od nalezeného indexu). Příklady pro vyhledávání nepravějšího výskytu:

- V poli: 1,2,3,4,5,5,6,6,6,8,9,13 najde algoritmus klíč **K=6** na pozici **8** (počítáno od 0).
- V poli: 1,1,1,1,1,1,1,1,1,2 najde algoritmus klíč **K=1** na pozici **9**.

Binární vyhledávání v binárním vyhledávacím stromu (binary search tree)

Podobné binárnímu vyhledávání v seřazeném poli. Je-li **vyhledaný klíč roven kořeni**, vyhledávání končí **úspěšně**. Je-li **klíč menší než klíč kořene**, pokračuje vyhledávání v **levém podstromu**, je-li **větší**, pokračuje v **pravém podstromu**. Vyhledávání končí **neúspěšně**, je-li **prohledávaný**

(pod)strom prázdný. Výhodou uspořádání do BVS je **snazší vkládání a mazání prvků**. Při vkládání uzlu je uzel vložen na listovou úroveň (pokud se jedná o samovyvažující se strom může být nutné **provést rotace**, aby **bylo dodrženo vyvážení**, viz obrázek). U mazání je nutné dodržet uspořádání stromu a pokud je mazán vnitřní uzel, musí být nahrazen některým uzlem z listové úrovně takto:

- **nejpravější** uzel **levého podstromu** rušeného uzlu (**maximum v levém podstromu**),
- **nejlevější** uzel **pravého podstromu** rušeného uzlu (**minimum v pravém podstromu**).

[[media/szz-30/media/image9.png]]

[[media/szz-30/media/image13.png]]

BVS se zarážkou

Funguje podobně jako u pole, všechny listové uzly ukazují na zarážku. Na konci vyhledávání musí být provedený test, jestli se vyhledala hledaná hodnota nebo zarážka.

Vyhledávání pomocí stromů s více klíči ve vrcholech

[[media/szz-30/media/image1.png]]

Vnitřní (interní) vrcholy obsahují libovolný nenulový počet klíčů. Pokud ve vrcholu (uzlu) leží **k** klíčů, pak má **k+1** synů (**s₀, ..., s_k**). Hledání cesty v uzlu se provádí obvykle sekvenčně/binárně (už nelze rozlišit pouze pravý/levý podstrom). **Sníží se hloubka** stromu ale **zvyšuje se náročnost hledání cesty**. **Vnitřní uzly nemusí nést data** a slouží pouze pro vyhledávání, data jsou uložena až na listové úrovni (**B+ stromy**, které se využívají v DB, file systémech, ...). Počet klíčů ve vrcholech se většinou může pohybovat od do určitého počtu - např. **(a,b) stromy** (kořen má **2** až **b** synů, ostatní vnitřní vrcholy **a** až **b** synů, **a** ≥ 2 , **b** $\geq 2a-1$). Při **vkládání** se uzly **štěpí** (je-li potřeba), při **mazání** se **slučují** (je-li potřeba). Slovník lze například implementovat pomocí **písmenkového stromu** (neboli **trie**), kde se každý uzel větví dle počtu písmen v abecedě, což umožňuje sdílet části stromů mezi více slov a vyhledání je velmi rychlé.

[[media/szz-30/media/image2.png]]

Vyhledávání v tabulkách s rozptýlenými položkami (TRP - hash table)

Základem je princip **tabulky s přímým přístupem**. Využívá se zde mapovací funkce, která **rovnoměrně mapuje klíče na indexy** v paměti (políčka tabulky). Což v ideálním případě bez kolizí znamená **konstantní O(1)** časovou složitost vyhledávání. Problém je vznik kolizí (klíče, které jsou mapovány na stejný index), poté musí být vyhledávání dokončeno sekvenčně. V **extrémním případě** tak může být rychlost vyhledávání **lineární O(n)**. Vyhledávání v TRP má **index-sekvenční** charakter.

- **explicitní** řetězení synonym: spojový seznam, binární vyhledávací strom,
- **implicitní** řetězení synonym: ukládání synonym na první/další volné místo (dle kroku) v tabulce (poli).

Vyhledávání v textu

Hledaným klíčem je řetězec (několik znaků) textu.

Naivní (Brute-force) algoritmus

Algoritmus **porovnává** symboly textu a vzorku **zleva doprava**, při **neshodě** symbolů **posune** vzorek **o jednu** pozici doprava.

- složitost: **$O(m \cdot n)$** , kde **m** je počet znaků textu a **n** je počet znaků vzorku, jedná se ale o **nejhorší případ** a většinou je porovnání daleko rychlejší.

Knuth-Morris-Prattův algoritmus

Algoritmus využívá princip **konečného automatu**. Porovnávání textu stále probíhá **zleva doprava**. Při neshodě se nevrací v textu zpět (posun o 1 v předchozím algoritmu), ale snaží se **posunout vzorek** tak, aby symbol textu, na kterém došlo k neshodě, porovnal s **jiným symbolem vzorku**, který předchází symbolu s neshodou (pokud žádný takový neexistuje, posouvá vzorek na začátek).

[[media/szz-30/media/image14.png]]

- složitost **$O(m)$** pro konstrukci automatu, **$O(n)$** pro porovnání, celkově **$O(m+n)$** .

Boyer-Mooreův algoritmus

Přikládá vzorek k textu **zleva doprava**, **ALE** porovnává **zprava doleva**, což umožňuje provádět větší skoky (některé **symboly textu nemusí být vůbec porovnány** se symboly vzorku). Pokud se při porovnávání narazí na **symbol**, který se ve vzorku **nenachází**, lze vzorek **posunout o jeho délku**, **jinak** je nutné vzorek **posunout** tak, aby byl **přiložen** na další **nejpravější výskyt** daného symbolu. Dále lze přidat **posun na základě podřetězce**.

Rabin-Karpův algoritmus

Vyhledávání vzorku založené na hashování. Postup:

- Posouváme okénko délky **m** (délky vzorku) po textu a **počítáme hash** pro danou část textu.
- Je-li **hash shodný s hashem vzorku**, **porovnáme** danou část textu se vzorkem **znak po znaku**.

Aby tato metoda byla efektivní, musíme umět **rychle počítat hash** (v konstantním čase). Lze zařídit tak, že znak, který **opouštíme**, od hashe určitým způsobem **odečteme** a znak, na který **najíždíme**, určitým způsobem **přičteme** (Ordinální hodnota asi nebude úplně efektivní ale bude taky fungovat, většinou jsou ale znaky násobeny nějakým **polynomem**).

Písmenkové stromy - hledání více vzorků

Umožňují současné vyhledávání více vzorků v textu. Na jedné cestě od kořene se může nacházet více vzorků (na obrázku jsou konce slov označeny hvězdičkou).

Algoritmus Aho-Corasicková

[[media/szz-30/media/image10.png]]

- Postupujeme automatem po dopředných hranách, pokud můžeme.
- Nelze-li použít žádnou dopřednou hranu, vracíme se po zpětných hranách.
- Pokud se dostaneme zpět až do kořene a ani zde nelze jít s daným symbolem žádnou dopřednou hranou, symbol je zahozen.
- V každém stavu zkontrolujeme, zda neodpovídá konci slova. Pokud ano, ohlásíme výskyt. Z každého stavu pomocí zkratk nalezneme také všechny sufixy, které jsou také slovem a ohlásíme.

[[media/szz-30/media/image7.png]]

odkazy

- Animace řazení: <https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

Zdroje

- SZZ okruh 30 — studijní materiály FIT BUT (szz-30.docx). Obrázky: media/szz-30/.

Navigace: [\[\[index| • Přehled okruhů\]\]](#) · [\[\[okruhy/29-datove-ridici-struktury|29. Datové a řídicí struktury\]\]](#) · další: [\[\[okruhy/31-pravdepodobnost-statistika|31. Pravděpodobnost a statistika\]\]](#) [\[\[okruhy/32-robotika|32. Robotika\]\]](#)